FIT2014 Practice Exam (seed 42) — Solutions

FIT2014 Theory of Computation · Total marks: 120 · Time: 3 hours

Question 1 (4 marks)

Topic: Propositional logic · source: procedural · modelled on exam-2014-q1

Your phone is somewhere in your house. Suppose we have propositions B, G, C, L with the following meanings.

- B: Your phone is in the bedroom.
- G: Your phone is in the garage.
- C: Your phone is in the bathroom.
- L: Your phone is in the laundry.

Use B, G, C, L to write a proposition, in Conjunctive Normal Form, that is True precisely when at least one of these 4 propositions is True.

Model solution:

(B ∨ G ∨ C ∨ L)

Here the clauses of size 4 ensure at least 1 are true.

Marking rubric:

- Logically correct (same truth table) — 3
- In Conjunctive Normal Form — 1

Question 2 (4 marks)

Topic: Regular expressions · source: procedural · modelled on sampleexam2013-q4

Consider the regular expression (ba∪a*)(ba∪ε) over the alphabet {a,b}.

(a) Write down all strings of at most 6 letters that match this regular expression. *[2 marks]*

Model solution:

ε, a, aa, ba, aaa, aba, aaaa, aaba, baba, aaaaa, aaaba, aaaaaa, aaaaba

Marking rubric:

- All strings listed, no extras — 2

(b) Give an NFA that recognises the language described by this regular expression. *[2 marks]*

Model solution:

One possibility (Thompson construction, simplified forms are equally acceptable):

```
start: 1
final: 1 2 5 7
1 a 2
```

```
1 b 3
2 a 2
2 b 4
3 a 5
4 a 7
5 b 4
```

Marking rubric:

- NFA accepts exactly the language of the regular expression — 2

Question 3 (4 marks)

Topic: Regular expressions · source: procedural · modelled on exam-2013-q3

Consider the regular expression ((ab)∪*(bab))* over the alphabet {a,b}.

(a) Write down all strings of at most 6 letters that match this regular expression. *[2 marks]*

Model solution:

ε, ab, bab, abab, abbab, babab, ababab, babbab

Marking rubric:

- All strings listed, no extras — 2

(b) Give an NFA that recognises the language described by this regular expression. *[2 marks]*

Model solution:

One possibility (Thompson construction, simplified forms are equally acceptable):

```
start: 1
final: 1
1 a 2
1 b 3
2 b 1
3 a 2
```

Marking rubric:

- NFA accepts exactly the language of the regular expression — 2

Question 4 (3 marks)

Topic: Closure properties of regular languages · source: past:exam-2013-q4 · modelled on exam-2013-q4

A language over alphabet {a,b} is said to be cofinite if it contains all strings over that alphabet except for some finite number of strings. Prove that every cofinite language is regular.

Model solution:

A cofinite language has a finite complement. Any finite language is regular. Regular languages are closed under complement. So any cofinite language (being the complement of a regular language)

is regular. Let L be any cofinite language. Its complement, L, is finite. Therefore L is regular, since every finite language is regular. Therefore, L is recognised by a Finite Automaton, by Kleene's Theorem. If we change that FA so that every Final state becomes Non-Final, and every Non-Final state becomes Final, then we obtain an FA that accepts precisely those strings not in L — in other words, this FA recognises L. Therefore L is regular, by Kleene's Theorem.

Marking rubric:

- Correct answer (compare with the official solution) — 3

Question 5 (7 marks)

Topic: Kleene's theorem / conversions · source: procedural · modelled on exam-2013-q5

The following table describes an NFA with ε-transitions (each cell lists the set of possible next states). Convert it to a Finite Automaton that recognises the same language, using the subset construction. Represent your FA as a transition table.

	state	a	b	ε
Start	1	{3,5}	∅	∅
	2	∅	{3}	∅
	3	∅	{4}	{2}
Final	4	{4}	∅	∅
	5	{5}	{5}	∅

Model solution:

Subset construction (each FA state is the ε-closure of a set of NFA states):

	state	a	b
Start	{1}	{2,3,5}	∅
	{2,3,5}	{5}	{2,3,4,5}
	∅	∅	∅
	{5}	{5}	{5}
Final	{2,3,4,5}	{4,5}	{2,3,4,5}
Final	{4,5}	{4,5}	{5}

Merging equivalent states (state minimisation) is also acceptable.

Marking rubric:

- Correct Start state (ε-closure of the NFA start state) — 1
- Correct transitions for every state — 5
- Correct Final states — 1

Question 6 (3 marks)

Topic: Regular expressions · source: procedural · modelled on exam-2014-q3

Give a regular expression for the language of all strings over {a,b} that contain aba as a substring.

Model solution:

(a∪b)*aba(a∪b)*

Marking rubric:

- Correct answer — 3

Question 7 (6 marks)

Topic: Context-free grammars · source: past:sampleexam2013-q10 · modelled on sampleexam2013-q10

Consider the following Regular grammar: S → ε | aT | bU (1) T → aT | bS (2) U → aS (3)

(a) Give (i) a derivation, and (ii) a parse tree, for the string baab . *[2 marks]*

Model solution:

b U a S S ⇒ bU by (1c) ⇒ baS by (3) a T ⇒ baaT by (1b) ⇒ baabS by (2b) b S ⇒ baab by (1a) ε

Marking rubric:

- Correct answer (compare with the official solution) — 2

(b) Find the Finite Automaton for the language defined by the above grammar. *[2 marks]*

Model solution:

a a T T a a b b S S a,b a a b b b U U

Marker comments: The FA on the left is obtained by direct conversion of the regular grammar to a FA. Strictly speaking, it is not deterministic since state U does not have a transition for the letter b. This is easily fixed: see the FA on the right. (In a sense, omitting transitions is a less “serious” form of nondeterminism than empty transitions or multiple transitions for the same letter, since the former does not lead to ambiguity as to how to proceed, provided a missing transition is taken to give rejection if it is attempted. That’s not how we’ve defined rejection by FAs in this unit, though.)

Marking rubric:

- Correct answer (compare with the official solution) — 2

(c) Give a regular expression for the language defined by the above grammar. *[2 marks]*

Model solution:

$((aa^*b) \cup (ba))^*$

Marking rubric:

- Correct answer (compare with the official solution) — 2

Question 8 (2 marks)

Topic: Context-free grammars · source: procedural · modelled on exam-2013-q8

Give a Context-Free Grammar for the language $\{ a^m b^n : m < n \}$.

Model solution:

S → aSb | Sb | b

Marking rubric:

- Generates every string in the language — 1
- Generates no strings outside the language — 1

Question 9 (6 marks)

Topic: Proof by induction · source: past:exam-2013-q9 · modelled on exam-2013-q9

Consider the following Context-Free Grammar S → a S (1) S → S b (2) S → ε (3) Prove by induction that every string of the form $a^m b^n$, where m ≥ 0 and n ≥ 0, can be generated by this grammar.

Model solution:

We prove this by induction on the length of the string (i.e., on m + n). Inductive basis: if the length is 0, then the string is empty, and is generated by (3). Inductive step: Suppose the result holds for all strings of length < ℓ, where ℓ> 0. Now suppose we have a string w = $a^m b^n$ of length ℓ. Either the string starts with a, or ends with b, or both. (This uses ℓ> 0.) If it starts with a, then it can be written w = aw′, where |w′| = |w| − 1 = ℓ − 1 < ℓ. The fact that w′ has length < ℓ means we can use the inductive hypothesis to conclude that w′ can be generated by the grammar. So we have a derivation S ⇒· · · ⇒ w′. By prefixing each string in this derivation by a, we can form another derivation: aS ⇒· · · ⇒ aw′ = w. Prepending this derivation with the production S ⇒ aS gives a derivation of w: S ⇒ aS ⇒· · · ⇒ aw′ = w. So w is generated by the grammar. A similar argument applies if the string w ends with b. This completes the inductive step. The result follows, by the Principle of Mathematical Induction.

Marking rubric:

- Correct answer (compare with the official solution) — 6

Question 10 (5 marks)

Topic: Context-free grammars · source: past:exam-2013-q10 · modelled on sampleexam2013-q13

Consider the following Context-Free Grammar. S → aBa (1) B → BB (2) B → Q (3) B → R (4) Q → q (5) R → r (6) Give

(a) a derivation, and *[3 marks]*

Model solution:

(no official solution available for this past question)

Marking rubric:

- Correct answer (compare with the official solution) — 3

(b) a parse tree, for the string aqrqa, labelling each step in the derivation on its right by the number of the rule used. Use the spaces below for your answers. (a) Derivation: (b) Parse tree: *[2 marks]*

Model solution:

S ⇒ aBa by (1) ⇒ aBBa by (2) ⇒ aBBBa by (2) ⇒ aQBBa by (3) ⇒ aQRBa by (4) ⇒ aQRQa by (3) ⇒ aqRQa by (5) ⇒ aqrQa by (6) ⇒ aqrqa by (5). S S a B a a B a B B B B B B B B Q Q Q R R Q q q q r r q

Marker comments: Some variation in the order is possible. Doesn't have to be leftmost or rightmost derivation. OR

Marking rubric:

- Correct answer (compare with the official solution) — 2

Question 11 (5 marks)

Topic: Context-free grammars · source: past:sampleexam2013-q11 · modelled on exam-2013-q11

A string over the alphabet {0,+,–} is said to be balanced if it satisfies both the following: • (i) for any i, the first i characters in the string contain at least as many + as –; • (ii) the whole string has the same number of + as –. Give a Context-Free Grammar for the language.

Model solution:

S → 0 S → SS S → +S – S → ε

Marking rubric:

- Correct answer (compare with the official solution) — 5

Question 12 (4 marks)

Topic: Context-free grammars · source: past:exam-2013-q12 · modelled on exam-2013-q12

State two important results that can be proved using the Chomsky Normal Form for Context- Free Grammars.

Model solution:

1. The CYK algorithm, which determines, for any CFG and any string, whether the string is generated by the CFG.
2. The Pumping Lemma for CFLs: for any context-free language L and any sufficiently long string $w \in L$, there exist u, v, x, y, z such that w = uvxyz, v and y are not both empty, |vxy| is small enough, and for all i, $uv^ixy^iz \in L$.

Marking rubric:

- Correct answer (compare with the official solution) — 4

Question 13 (6 marks)

Topic: Turing machines · source: procedural · modelled on exam-2013-q13

Draw a Turing machine that adds 1 to a binary number (most significant bit first). If the input is all 1s, the output is one bit longer, e.g. 111 becomes 1000. The input is a string over {0,1}, starting in the leftmost cell with the head on it. For example, on input 1011 the output is 1100.

Model solution:

One possible machine:

```
start: 0
```

```
halt: H
# mark the first cell (X = 0, Y = 1) so the left end can be recognised
0 0 -> 1 X R
0 1 -> 1 Y R
0 Δ -> H 1 R
1 0 -> 1 0 R
1 1 -> 1 1 R
1 Δ -> 2 Δ L
2 1 -> 2 0 L
2 0 -> 5 1 L
2 X -> H 1 R
2 Y -> 4 1 R
4 0 -> 4 0 R
4 Δ -> H 0 R
5 0 -> 5 0 L
5 1 -> 5 1 L
5 X -> H 0 R
5 Y -> H 1 R
```

Marking rubric:

- Produces the correct output on all inputs — 5
- Halts properly (does not crash or loop) — 1

Question 14 (4 marks)

Topic: Church-Turing thesis · source: past:exam-2013-q14 · modelled on exam-2013-q14

(a) State the Church-Turing thesis. *[2 marks]*

Model solution:

Any function which can defined by an algorithm can be represented by a Turing Machine.

Marking rubric:

- Correct answer (compare with the official solution) — 2

(b) Give two reasons why the Church-Turing thesis is widely accepted. *[2 marks]*

Model solution:

Any two of the following: • different approaches to defining computability end up being equivalent • long experience, that algorithms can be implemented as programs, and therefore on Turing machines • no known counterexamples, i.e., no algorithms which seem to be unimplementable

Marking rubric:

- Correct answer (compare with the official solution) — 2

Question 15 (4 marks)

Topic: Decidability / undecidability · source: procedural · modelled on sampleexam2013-q18

For each of the following decision problems, indicate whether it is Decidable or Undecidable.

(a) Input: a Turing machine M.
Question: Does M ever write the symbol a on its tape, when run on the empty string, within its first 1000 steps?

(b) Input: a Turing machine M and a string w.
Question: If M is run on w, does it halt within 100 steps?

(c) Input: a Turing machine M.
Question: Is the language accepted by M finite?

(d) Input: a Turing machine M.
Question: Does M halt when given the empty string as input?

(a)

Model solution:

Decidable

Marking rubric:

- Correct classification — 1

(b)

Model solution:

Decidable

Marking rubric:

- Correct classification — 1

(c)

Model solution:

Undecidable

Marking rubric:

- Correct classification — 1

(d)

Model solution:

Undecidable

Marking rubric:

- Correct classification — 1

Question 16 (10 marks)

Topic: Classes of languages · source: procedural · modelled on exam-2013-q16

Consider the classes of languages Finite $\subset$ Regular $\subset$ Context-Free $\subset$ P $\subseteq$ NP $\subset$ Decidable $\subset$ Recursively Enumerable (r.e.). For each language below, state the **smallest** of these classes that it is known to belong to, or **None** if it is not r.e. (Assume P $\neq$ NP.)

(a) The set of all graphs containing no circuit.
(b) The set of all correctly formed arithmetic expressions using positive integers, +, – and parentheses.
(c) The set of all (encodings of) Turing machines that accept every string.
(d) The set of binary representations of prime numbers.
(e) The language of all strings over {a,b} that do not contain bb as a substring
(f) The language of positive integers in binary with leading bit 1
(g) The language of binary strings (possibly with leading zeros) that represent a multiple of 3
(h) The set of all connected graphs.
(i) The language { ww : w $\in$ {a,b}^* }
(j) The language of strings over {a,b} with more a's than b's

(a)

Model solution:

P

Marking rubric:

- Correct class — 1

(b)

Model solution:

Context-Free

Marking rubric:

- Correct class — 1

(c)

Model solution:

None

Marking rubric:

- Correct class — 1

(d)

Model solution:

P

Marking rubric:

- Correct class — 1

(e)

Model solution:

Regular

Marking rubric:

- Correct class — 1

(f)

Model solution:

Regular

Marking rubric:

- Correct class — 1

(g)

Model solution:

Regular

Marking rubric:

- Correct class — 1

(h)

Model solution:

P

Marking rubric:

- Correct class — 1

(i)

Model solution:

P

Marking rubric:

- Correct class — 1

(j)

Model solution:

Context-Free

Marking rubric:

- Correct class — 1

Question 17 (7 marks)

Topic: Pumping lemma (context-free) · source: procedural · modelled on exam-2014-q12

Use the Pumping Lemma for Context-Free Languages to prove that the language $\{ a^n b^n c^n : n \geq 0 \}$ is not context-free.

Model solution:

Suppose that the language $L = \{ a^n b^n c^n : n \geq 0 \}$ is context-free. Then there exists a context-free grammar that generates L, or equivalently, there exists a pushdown automaton (PDA) that accepts L. Let N be the number of states in this PDA. Let $w = a^N b^N c^N$, which is a string in L and has length $3N \geq N$. By the Pumping Lemma for context-free languages, w can be written as w = uvxyz, where:

1. $|vxy| \leq N$,
2. $|vy| \geq 1$,
3. For all $i \geq 0$, $uv^{ixyiz} \in L$.

We consider the possible cases for the substring vxy:

Case 1: vxy contains only a's. Then, pumping v or y increases the number of a's without affecting the number of b's or c's. For example, uv^{2xyiz} would have more a's than b's or c's, which is not in L.

Case 2: vxy contains only b's. Then, pumping v or y increases the number of b's without affecting the number of a's or c's. For example, uv^{ixy2z} would have more b's than a's or c's, which is not in L.

Case 3: vxy contains only c's. Then, pumping v or y increases the number of c's without affecting the number of a's or b's. For example, uv^{ixyiz} would have more c's than a's or b's, which is not in L.

Case 4: vxy contains a mix of a's and b's. Then, pumping v or y increases the number of a's or b's without affecting the number of c's. For example, uv^{ixyiz} would have more a's than b's or more b's than c's, which is not in L.

Case 5: vxy contains a mix of b's and c's. Then, pumping v or y increases the number of b's or c's without affecting the number of a's. For example, uv^{ixyiz} would have more b's than c's or more c's than a's, which is not in L.

Case 6: vxy contains a mix of a's and c's. Then, pumping v or y increases the number of a's or c's without affecting the number of b's. For example, uv^{ixyiz} would have more a's than c's or more c's than b's, which is not in L.

In all cases, pumping leads to a string not in L, which contradicts the Pumping Lemma. Hence, our assumption that L is context-free is false. Therefore, L is not context-free.

Marking rubric:

- State the Pumping Lemma for context-free languages — 2
- Choose a suitable string $w = a^N b^N c^N$ and justify its length — 1
- Apply the Pumping Lemma and consider all possible cases for vxy — 3
- Show that pumping leads to a string not in L in all cases — 1

Question 18 (5 marks)

Topic: Decidability / undecidability · source: procedural · modelled on exam-2014-q14

For each of the following decision problems, indicate whether it is Decidable or Undecidable.

(a) Input: two Turing machines M1 and M2.
Question: Do M1 and M2 accept the same language?

(b) Input: a Turing machine M.
Question: Does M ever move its head left, when run on the empty string, during its first 50 steps?

(c) Input: a Context-Free Grammar G and a string w.
Question: Does G generate w?

(d) Input: two Finite Automata A and B.
Question: Do A and B accept the same language?

(e) Input: a Turing machine M.
Question: Is the language accepted by M finite?

(a)

Model solution:

Undecidable

Marking rubric:

- Correct classification — 1

(b)

Model solution:

Decidable

Marking rubric:

- Correct classification — 1

(c)

Model solution:

Decidable

Marking rubric:

- Correct classification — 1

(d)

Model solution:

Decidable

Marking rubric:

- Correct classification — 1

(e)

Model solution:

Undecidable

Marking rubric:

- Correct classification — 1

Question 19 (4 marks)

Topic: Classes of languages · source: procedural · modelled on exam-2017-q13

Consider the classes of languages Finite $\subset$ Regular $\subset$ Context-Free $\subset$ P $\subseteq$ NP $\subset$ Decidable $\subset$ Recursively Enumerable (r.e.). For each language below, state the **smallest** of these classes that it is known to belong to, or **None** if it is not r.e. (Assume P $\neq$ NP.)

(a) The set of all Context-Free Grammars that generate the empty language.
(b) The language of binary strings (possibly with leading zeros) that represent a multiple of 3
(c) The language of all strings over {a,b} that do not contain bb as a substring
(d) The language of all strings over {a,b} whose first and last letters are different

(a)

Model solution:

P

Marking rubric:

- Correct class — 1

(b)

Model solution:

Regular

Marking rubric:

- Correct class — 1

(c)

Model solution:

Regular

Marking rubric:

- Correct class — 1

(d)

Model solution:

Regular

Marking rubric:

- Correct class — 1

Question 20 (4 marks)

Topic: Decidability / undecidability · source: past:sampleexam2013-q18 · modelled on sampleexam2013-q18

For each of the following decision problems, indicate whether or not it is decidable. You may assume that, when Turing machines are encoded as strings, this is done using the Code-Word Language (CWL). Decision Problem your answer (tick one box in each row) Input: Turing machines M and N. Question: Are the encoded forms of M and N identical? Decidable Undecidable Input: Turing machines M and N. Question: Do M and N have the same time complexity? Decidable Undecidable Input: a Turing machine M. Question: Does M correctly determine whether or not its input string is a palindrome? Decidable Undecidable Input: a Turing machine M, and a string w. Question: Does M ever change any letter of w on the tape? Decidable Undecidable

Model solution:

✓ ✓ ✓ ✓

Marking rubric:

- Correct answer (compare with the official solution) — 4

Question 21 (2 marks)

Topic: Time complexity / class P · source: past:exam-2013-q20 · modelled on exam-2013-q21

Prove that the class of regular languages is a subset of the class P.

Model solution:

Let L be a regular language. Then, by Kleene's Theorem, there is a FA which recognises L. We can turn this into a Turing machine (or a computer program) that keeps track of which state the FA is in, at any time, and depending on that state, and what the next letter is, determines what the next state is. This TM (or program) looks at each letter of the input string, in turn, and the work it does for that letter — checking what it is and changing its record of the current state accordingly — can be done in only a few steps (constant time or linear time, depending on how the information is organised and managed). Multiply this by the number of letters in the string, and you get an overall time complexity of quadratic time or better. So it takes polynomial time, so L is in P. Hence every regular language is in P.

Marking rubric:

- Correct answer (compare with the official solution) — 2

Question 22 (21 marks)

Topic: NP and NP-completeness · source: past:exam-2013-q22 · modelled on exam-2013-q22

Consider the language NO MONOCHROMATIC TRIANGLE, which consists of all graphs G such that we can assign colours to the edges of the graph so that (i) each edge is either Black or White,

and (ii) there is no triangle (i.e., cycle of length 3) in the graph which is monochromatic (i.e., all its edges have the same colour).

(a) Prove that the language NO MONOCHROMATIC TRIANGLE is in NP. Now, let W be the following graph. b y y

c a e

y y d *[4.0 marks]*

Model solution:

Here is a verifier: Input: graph G. Certificate: a 2-colouring of the edges of G. Verification: For each triangle of G: { if the three edges of the triangle all have the same colour, Reject and Stop. } Accept and Stop. [If we reach this point, each triangle must have been non-monochromatic.] It should be clear from the construction of the verifier that it will accept if and only if the input graph belongs to NO MONOCHROMATIC TRIANGLE. It remains to show that this verifier runs in polynomial time. To see this, observe that a graph on n vertices has $O(n^3)$ triangles, so the number of outer-loop iterations is $O(n^3)$. For each such triangle, we must check that the three edges do not all receive the same colour. The exact amount of time taken by this depends on the details of the implementation (what data structure is used for the graph, etc.), but in any case takes polynomial time, and should take constant or linear time. Multiplying this by $O(n^3)$ gives a total time which is polynomial in the input size. So the verifier is a polynomial-time verifier. So NO MONOCHROMATIC TRIANGLE has a polynomial-time verifier, hence it is in NP.

Marker comments: [This can be described in various ways. E.g., as an assignment of either Black or White to each edge of G, or a list of colours, Black/White, one colour for each edge, or a partition of the set of edges into two parts, etc.]

Marking rubric:

- Correct answer (compare with the official solution) — 4.0

(b) Construct a Boolean expression E_{W} in Conjunctive Normal Form such that the satis- fying truth assignments for E_{W} correspond to solutions to the NO MONOCHROMATIC TRIANGLE problem on the above graph W (i.e., they correspond to colourings of the edges of W which have no monochromatic triangle). *[6.0 marks]*

Model solution:

Using variables a, b, c, d, e, with True meaning Black and False meaning White (or the other way round): (a ∨ b ∨ c) ∧ (¬a ∨¬b ∨¬c) ∧ (c ∨ d ∨ e) ∧ (¬c ∨¬d ∨¬e) An alternative approach is to use variables B_{a}, W_{a}, B_{b}, W_{b}, B_{c}, W_{c}, B_{d}, W_{d}, B_{e}, W_{e}, with the interpretation that, for any edge x in the graph, B_{x} = True if edge x is Black, and False otherwise; W_{x} = True if edge x is White, and False otherwise. This is more cumbersome. The expression should then be: (B_{a} ∨ B_{b} ∨ B_{c}) ∧ (W_{a} ∨ W_{b} ∨ W_{c}) ∧ (B_{c} ∨ B_{d} ∨ B_{e}) ∧ (W_{c} ∨ W_{d} ∨ W_{e}) ∧ (Ba ∨ Wa) ∧ (¬Ba ∨¬Wa) ∧ (Bb ∨ Wb) ∧ (¬Bb ∨¬Wb) ∧ (Bc ∨ Wc) ∧ (¬Bc ∨¬Wc) ∧ (Bd ∨ Wd) ∧ (¬Bd ∨¬Wd) ∧ (Be ∨ We) ∧ (¬Be ∨¬We) The first four clauses are clearly analogous to the four clauses of the earlier expression. The last ten clauses are to ensure that each edge gets precisely one colour.

Marking rubric:

- Correct answer (compare with the official solution) — 6.0

(c) Give a polynomial-time reduction from NO MONOCHROMATIC TRIANGLE to SAT-ISFIABILITY. *[7.0 marks]*

Model solution:

Input: Graph G. For each edge of G, create a new variable. For each triangle T of G: { Let a, b, c be the edges of T. Create two new clauses: a ∨ b ∨ c, ¬a ∨¬b ∨¬c. } Output: the conjunction of all the clauses created so far.

Marker comments: [Some might use the more cumbersome set of clauses, but that's fine if done right.]

Marking rubric:

- Correct answer (compare with the official solution) — 7.0

(d) State the Cook-Levin Theorem. *[2.0 marks]*

Model solution:

SATISFIABILITY is NP-complete.

Marking rubric:

- Correct answer (compare with the official solution) — 2.0

(e) Given the facts stated so far in this question: What else, if anything, would you need to prove, in order to show that NO MONOCHRO- MATIC TRIANGLE is NP-complete? *[2.0 marks]*

Model solution:

SAT ≤_{P} NO MONOCHROMATIC TRIANGLE.

Marker comments: Reasoning, though the question doesn't ask for it: We know SAT is NP-complete, and we know NO MONOCHROMATIC TRIANGLE is in NP. So, if we show that SAT ≤_{P} NO MONOCHROMATIC TRIANGLE, then we have completed the proof that NO MONOCHROMATIC TRIANGLE is NP-complete.

Marking rubric:

- Correct answer (compare with the official solution) — 2.0